In these situations, you can define the hooks once for your entire suite, in an RSpec.configure block (typically in spec/spec_helper.rb or somewhere in spec/support):

07-structuring-code-examples/04/spec/spec_helper.rb

```
RSpec.configure do |config|
  config.around(:example) do |ex|
    original_env = ENV.to_hash
    ex.run
    ENV.replace(original_env)
  end
end
```

You've defined these *config hooks* in one place, but they'll run for *every* example in your test suite. Note the trade-offs here:

- Global hooks reduce duplication, but can lead to surprising "action at a distance" effects in your specs.[3]
- Hooks inside example groups are easier to follow, but it's easy to leave out an important hook by mistake when you're creating a new spec file.

Use Config Hooks for Incidental Details



We recommend you only use config hooks for things that aren't essential for understanding how your specs work. The bits of logic that isolate each example—such as database transactions or environment sandboxing—are prime candidates.

We prefer to keep things simple and run our hooks unconditionally. If, however, our config hooks are only needed by a subset of examples—and particularly if they are slow—we'll use metadata to make sure they run only for the subset that need them. You've already used this technique in *Isolating Your Specs Using Database Transactions*, on page 92.

Scope

Most of the hooks you've written are meant to run once per example. After all, you want to make sure your examples can run in any order and that any individual example can run on its own. Since this behavior is the norm, RSpec sets the scope to :example if you do not provide one.

Sometimes, though, a hook needs to do a really time-intensive operation like creating a bunch of database tables or launching a live web browser. Running the hook once per spec would be cost-prohibitive.

3. https://en.wikipedia.org/wiki/Action_at_a_distance_(computer_programming)